POKESISDIS

Sistema de Gestión de Excepciones Distribuidas

Práctica Obligatoria 2

Asignatura: Sistemas Distribuidos
Titulación: 4 Grado en Ingeniería Informática
Tecnologías: Spring Boot + Flask (Python) + PostgreSQL + Docker
Frontend: Thymeleaf + Spring Security
Infraestructura: Docker Compose
Alumno: Juan Luis Fernández Bayona

Índice de Contenidos

Índice de Contenidos 2
1. Introducción y Objetivo 3
Objetivos específicos de la práctica: 3
2. Arquitectura del Sistema 4
3. Tecnologías Utilizadas 4
4. Estructura del Proyecto 5
5. Componente 1: Spring Boot Frontend 6
5.1 Configuracion y Dependencias (pom.xml) 6
5.2 Capa de Seguridad (Spring Security) 6
5.3 Modelos de Datos (JPA + PostgreSQL) 6
5.4 Controladores Web 7
5.5 Manejador Global de Excepciones 7
Excepciones especificas del ApiSimulatorController: 7
6. Componente 2: API Flask Python 8
6.1 Excepciones de Archivos 8
6.2 Excepciones de Base de Datos 8
6.3 Excepciones de API Externa (PokeAPI) 8
7. Componente 3: PostgreSQL con Docker 9
Esquema de base de datos generado por Hibernate: 9
8. Orquestacion con Docker Compose 10
9. Flujo de Comunicacion y Manejo de Excepciones 11
Flujo de una peticion exitosa: 11
Flujo de una peticion con error controlado: 11
Flujo cuando Flask no esta disponible: 11
10. Guia de Arranque Completa 12
PASO 1 - Verificar que Docker Desktop esta corriendo 12
PASO 2 - Abrir terminal en la carpeta del proyecto 12
PASO 3 - Levantar Docker (PostgreSQL + Flask) 12
PASO 4 - Verificar contenedores activos 12
PASO 5 - Verificar que Flask responde 12
PASO 6 - Arrancar Spring Boot desde IntelliJ 12
PASO 7 - Verificar la aplicacion completa 12
Comandos Docker utiles: 13
11. Pruebas con Postman 13
Carpeta 1: Flask API Directa (localhost:5000) 13
Carpeta 2: Spring Boot a Flask (localhost:8080) 13
12. Verificacion y Comprobacion del Sistema 14
12.1 Verificacion de Docker 14
12.2 Verificacion del Frontend Spring Boot 14

12.3 Verificacion del Simulador de Excepciones 15
12.4 Verificacion de Logs en IntelliJ 15
12.5 Verificacion de Logs en Docker (Flask) 15
13. Conclusiones 16
Logros tecnicos alcanzados: 16
Aprendizajes de la practica: 16

1. Introducción y Objetivo

La Practica Obligatoria 2 de la asignatura Sistemas Distribuidos tiene como objetivo principal el diseño, implementación y puesta en marcha de un sistema distribuido completo centrado en la gestión de excepciones. El sistema demuestra cómo diferentes capas de una arquitectura distribuida pueden comunicarse, detectar errores y presentarlos al usuario de forma clara y comprensible, sin exponer detalles técnicos internos.

El sistema desarrollado recibe el nombre de PokeSisDis y consta de tres componentes independientes que colaboran entre sí: un frontend y orquestador construido con Spring Boot, una API REST desarrollada en Python con Flask, y una base de datos PostgreSQL. Todos los componentes externos al frontend se ejecutan dentro de contenedores Docker.

Objetivos específicos de la practica:

- Implementar un frontend web con Spring Boot, Thymeleaf y Spring Security con autenticación mediante login.
- Desarrollar una API REST en Python con Flask que cubra los tres tipos de excepciones requeridos: errores de archivos, errores de base de datos y errores de llamadas a APIs externas.
- Demostrar que las excepciones no críticas se capturan, traducen al español y se muestran al usuario de forma amigable sin romper la aplicación.
- Orquestar el entorno completo con Docker Compose, incluyendo PostgreSQL y la API Flask.
- Documentar y probar todos los endpoints con Postman, incluyendo casos de éxito y de error.
- Aplicar el patrón ControllerAdvice para el manejo centralizado de excepciones en Spring Boot.

2. Arquitectura del Sistema

El sistema sigue una arquitectura de tres capas distribuidas donde cada componente tiene una responsabilidad bien definida. La comunicación entre Spring Boot y Flask se realiza mediante peticiones HTTP usando RestTemplate. La comunicación entre Flask y PostgreSQL se realiza mediante el driver psycopg2.

Capa	Tecnología	Puerto	Entorno	Responsabilidad
Frontend / Orquestador	Spring Boot 3.4 + Thymeleaf	8080	Local (IntelliJ)	Interfaz, autenticación, llamadas a Flask
API REST	Flask 3.1 (Python 3.12)	5000	Docker	Lógica de excepciones: archivos, BD, PokeAPI
Base de Datos	PostgreSQL 16	5432	Docker	Persistencia de usuarios y roles
API Externa	PokeAPI (pokeapi.co)	443 HTTPS	Internet	Fuente de datos para excepciones de terceros

El usuario interactúa con el navegador accediendo a Spring Boot en el puerto 8080. Cuando se invoca un endpoint del simulador, Spring Boot realiza una petición HTTP a la API Flask en el puerto 5000 usando RestTemplate. Flask procesa la petición, pudiendo acceder a PostgreSQL o a la PokeAPI. Si ocurre una excepción, Flask la captura y devuelve un JSON estructurado. Spring Boot interpreta el código HTTP, traduce el mensaje al español y lo presenta al usuario vía Thymeleaf, sin mostrar jamás un stacktrace.

3. Tecnologías Utilizadas

Tecnología	Versión	Uso en el proyecto
Spring Boot	3.4.0	Framework principal del frontend y orquestador de peticiones
Spring Security	6.x	Autenticación y autorización con login/logout y BCrypt
Spring Data JPA	3.4.0	Acceso a datos con Hibernate y mapeo objeto-relacional
Thymeleaf	3.x	Motor de plantillas HTML para las vistas del frontend
PostgreSQL Driver	42.x	Conector JDBC para PostgreSQL (sustituye a MySQL)
Lombok	1.18.x	Reducción de código boilerplate en modelos Java
Flask	3.1.0	Microframework Python para la API REST de excepciones
psycopg2-binary	2.9.10	Driver Python para conexión a PostgreSQL
requests	2.32.3	Librería Python para llamadas HTTP a APIs externas
PostgreSQL	16 Alpine	Base de datos relacional en contenedor Docker
Docker	27.x	Contenerización de PostgreSQL y Flask
Docker Compose	v2	Orquestación multi-contenedor del entorno
Java	21 (LTS)	Lenguaje de programacion del frontend Spring Boot
Python	3.12	Lenguaje de programacion de la API Flask
Maven	3.9.x	Gestión de dependencias y build del proyecto Java
Postman	Actual	Herramienta para pruebas y documentación de APIs

4. Estructura del Proyecto

El proyecto se organiza en dos grandes bloques: el proyecto Spring Boot gestionado por Maven y la carpeta python-api con la API Flask. Ambos se encuentran en el mismo directorio raíz junto al archivo docker-compose.yml que controla el entorno.

```
practica2/
|-- pom.xml                              (dependencias Maven)
|-- docker-compose.yml                   (orquestacion Docker)
|
|-- python-api/                          (API Flask Python)
|   |-- app.py                           (logica de excepciones)
|   |-- requirements.txt                 (dependencias Python)
|   +-- Dockerfile                       (imagen Docker de Flask)
|
+-- src/main/java/com/sistdist/practica2/
    |-- Practica2Application.java        (entrada + datos iniciales)
    |-- config/
    |   |-- CustomUserDetailsService.java
    |   |-- SecurityConfig.java
    |   +-- RestTemplateConfig.java
    |-- controller/
    |   |-- MainController.java
    |   |-- ApiSimulatorController.java
    |   +-- GlobalExceptionHandler.java
    |-- dto/ApiResponseDto.java
    |-- model/   User.java  Role.java
    |-- repository/  UserRepository.java  RoleRepository.java
    +-- resources/
        |-- application.properties
        |-- templates/  (index, login, dashboard, api-simulator, error)
        +-- static/css/styles.css
```

5. Componente 1: Spring Boot Frontend

5.1 Configuración y Dependencias (pom.xml)

El proyecto Spring Boot utiliza Maven como gestor de dependencias. La diferencia más importante respecto a proyectos anteriores es la sustitución del driver MySQL por el driver PostgreSQL. Usamos Spring Boot 3.4.0 con Java 21 (LTS).

Dependencia	Propósito
spring-boot-starter-web	Servidor HTTP embebido y controladores MVC
spring-boot-starter-thymeleaf	Motor de plantillas para vistas HTML
thymeleaf-extras-springsecurity6	Integración de Spring Security en vistas (sec:authorize)
spring-boot-starter-security	Autenticación, autorización y formulario de login
spring-boot-starter-data-jpa	Acceso a datos con Hibernate y Spring Data
postgresql (runtime)	Driver JDBC para PostgreSQL en lugar de MySQL
lombok	Generación automática de getters, setters y constructores

```
# Fragmento de application.properties
spring.datasource.url=jdbc:postgresql://localhost:5432/sisdis_db
spring.datasource.username=sisdis
spring.datasource.password=sisdis123
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
python.api.base-url=http://localhost:5000
```

5.2 Capa de Seguridad (Spring Security)

La seguridad se implementa con Spring Security 6.x. Se definen dos componentes principales: CustomUserDetailsService, que carga los usuarios desde PostgreSQL en base al username, y SecurityConfig, que configura las reglas de acceso, el formulario de login y el logout.

URL / Patron	Acceso
/, /css/**, /login, /error	Publico (sin autenticación requerida)
/admin/**	Solo usuarios con rol ROLE_ADMIN
Resto de URLs	Requiere autenticación (cualquier usuario)

> **NOTA:** Las contraseñas se almacenan cifradas con BCryptPasswordEncoder. Al arrancar la aplicación, se insertan automáticamente dos usuarios de prueba: admin/admin123 (ROLE_ADMIN) y usuario/user123 (ROLE_USER).

5.3 Modelos de Datos (JPA + PostgreSQL)

Entidad	Tabla	Campos principales	Relación
Role	role	id, roleName, showOnCreate	Ninguna (lado 1)
User	users	id, username, email, password, role_id	ManyToOne con Role (EAGER)

> **NOTA:** IMPORTANTE: La tabla de usuarios se llama 'users' (no 'user') porque 'user' es una palabra reservada en PostgreSQL y causaría errores en la creación de la tabla.

5.4 Controladores Web

Controlador	Rutas	Responsabilidad
MainController	/, /login, /dashboard	Navegación principal. Gestiona mensajes de error y logout.
ApiSimulatorController	/api-simulator, /api-simulator/llamar	Invoca la API Flask via RestTemplate y captura todas las excepciones REST.
GlobalExceptionHandler	Todas (@ControllerAdvice)	Captura excepciones no controladas de acceso a datos y peticiones invalidas.

5.5 Manejador Global de Excepciones

El GlobalExceptionHandler está anotado con @ControllerAdvice, lo que le permite interceptar excepciones lanzadas por cualquier controlador. Garantiza que ningún error interno llegue sin tratar al usuario.

Excepción capturada	Causa típica	Acción
DataAccessException	PostgreSQL caído o query incorrecta	Muestra mensaje de BD, critico=false
NoHandlerFoundException	URL que no existe (404)	Muestra página de URL no encontrada
Exception (genérica)	Cualquier error no previsto	Muestra error genérico, critico=true

Excepciones específicas del ApiSimulatorController:

Excepción REST	Cuando ocurre	Mensaje al usuario
HttpClientErrorException	Flask devuelve 4xx (404, 403...)	Mensaje según código HTTP
HttpServerErrorException	Flask devuelve 5xx (500, 504...)	Mensaje según código HTTP
ResourceAccessException	Flask no disponible / Docker parado	'No se pudo conectar con la API Python'
RestClientException	Cualquier otro error REST	Mensaje de error genérico REST

6. Componente 2: API Flask Python

La API Flask es el núcleo de la practica desde el punto de vista de las excepciones. Implementa los tres tipos de excepciones, cada uno con sus variantes de éxito y de error. Todos los endpoints devuelven JSON estructurado con campos estandarizados: error, tipo y detalle.

6.1 Excepciones de Archivos

Se implementan dos endpoints para demostrar el manejo de errores de sistema de archivos. El endpoint de éxito crea el archivo si no existe y lo lee correctamente. El endpoint de error intenta leer un archivo que definitivamente no existe, forzando la excepción.

Endpoint	Método	Excepción que maneja	Código HTTP
/api/file-ok	GET	Ninguna (caso de éxito)	200 OK
/api/file-error	GET	FileNotFoundError	404 Not Found
/api/file-error	GET	PermissionError	403 Forbidden
/api/file-error	GET	OSError (genérico)	500 Internal Server Error

6.2 Excepciones de Base de Datos

Se implementan dos endpoints para demostrar el manejo de errores de acceso a PostgreSQL usando el driver psycopg2. El endpoint de éxito se conecta a la BD real en Docker y consulta la versión de PostgreSQL. El endpoint de error usa credenciales y host inválidos con un timeout de conexión de 3 segundos para que falle rápidamente.

Endpoint	Método	Excepción que maneja	Código HTTP
/api/db-ok	GET	Ninguna (conexión correcta)	200 OK
/api/db-error	GET	psycopg2.OperationalError (credenciales invalidas)	500
/api/db-error	GET	psycopg2.DatabaseError (error genérico de BD)	500

6.3 Excepciones de API Externa (PokeAPI)

Se implementan tres endpoints para demostrar el manejo de errores en llamadas a APIs de terceros. Se usa la PokeAPI como API externa real. El endpoint de timeout conecta a httpbin.org/delay/30 con un timeout de 2 segundos para forzar el error de tiempo de espera.

Endpoint	Método	Excepción	Código HTTP
/api/pokemon/pikachu	GET	Ninguna (éxito)	200 OK
/api/pokemon-error	GET	requests.HTTPError (404 de PokeAPI)	404
/api/pokemon-timeout	GET	requests.Timeout (2s agotados)	504 Gateway Timeout
/api/pokemon/<nombre>	GET	requests.ConnectionError	502 Bad Gateway
/api/stats	GET	Ninguna (estadísticas de sesión)	200 OK

7. Componente 3: PostgreSQL con Docker

PostgreSQL se ejecuta en un contenedor Docker usando la imagen oficial postgres:16-alpine. La base de datos se crea automáticamente con las credenciales definidas en las variables de entorno del docker-compose.yml.

Parámetro	Valor	Descripción
POSTGRES_DB	sisdis_db	Nombre de la base de datos creada automáticamente
POSTGRES_USER	sisdis	Usuario administrador de la base de datos
POSTGRES_PASSWORD	sisdis123	Contraseña del usuario
Puerto externo	5432	Accesible desde Spring Boot en localhost:5432
Puerto interno	5432	Accedido por Flask como postgres:5432 dentro de Docker
Volumen	pgdata	Persiste los datos entre reinicios del contenedor
Health check	pg_isready	Verifica que PostgreSQL acepta conexiones antes de arrancar Flask

Esquema de base de datos generado por Hibernate:

```
-- Tabla role (generada desde Role.java)
CREATE TABLE role (
    id             SERIAL PRIMARY KEY,
    role_name      VARCHAR(255) NOT NULL UNIQUE,
    show_on_create INTEGER NOT NULL
);

-- Tabla users (generada desde User.java)
-- NOTA: se llama "users" no "user" (palabra reservada en PostgreSQL)
CREATE TABLE users (
    id       SERIAL PRIMARY KEY,
    username VARCHAR(50)  NOT NULL UNIQUE,
    email    VARCHAR(100),
    password VARCHAR(250) NOT NULL,
    role_id  INTEGER REFERENCES role(id)
);

-- Datos iniciales insertados por Practica2Application.java al arrancar
INSERT INTO role VALUES (1, 'ROLE_ADMIN', 1), (2, 'ROLE_USER', 1);
INSERT INTO users VALUES (1,'admin','admin@sisdis.com','[bcrypt_hash]',1);
INSERT INTO users VALUES (2,'usuario','usuario@sisdis.com','[bcrypt_hash]',2);
```

8. Orquestación con Docker Compose

El archivo docker-compose.yml define y orquesta los dos servicios Docker: postgres y python-api. Spring Boot no se incluye en Docker porque se ejecuta desde IntelliJ para facilitar el desarrollo.

```
services:
  postgres:
    image: postgres:16-alpine
    container_name: sisdis_postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB:       sisdis_db
      POSTGRES_USER:     sisdis
      POSTGRES_PASSWORD: sisdis123
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U sisdis -d sisdis_db"]
      interval: 10s
      timeout: 5s
      retries: 5

  python-api:
    build: ./python-api
    container_name: sisdis_flask
    restart: unless-stopped
    environment:
      DATABASE_URL: postgresql://sisdis:sisdis123@postgres:5432/sisdis_db
    ports:
      - "5000:5000"
    depends_on:
      postgres:
        condition: service_healthy

volumes:
  pgdata:
```

NOTA: La cláusula depends_on con condition: service_healthy garantiza que Flask no arranca hasta que PostgreSQL este completamente listo. Esto evita errores de conexion durante el inicio del sistema.

9. Flujo de Comunicación y Manejo de Excepciones

Flujo de una petición exitosa:

1. Usuario pulsa botón en /api-simulator (p.ej. 'Consulta BD OK').
2. El navegador envía POST a Spring Boot: /api-simulator/llamar con endpoint=/api/db-ok.
3. ApiSimulatorController construye la URL: http://localhost:5000/api/db-ok.
4. Spring Boot llama a Flask con restTemplate.getForEntity(url, ApiResponseDto.class).
5. Flask se conecta a PostgreSQL con psycopg2 y obtiene la versión de la BD.
6. Flask devuelve HTTP 200 con JSON: {"resultado": "Conexion correcta a PostgreSQL 16..."}.
7. Spring Boot recibe la respuesta y puebla el modelo (éxito=true, resultado, statusCode=200).
8. Thymeleaf renderiza el panel verde en api-simulator.html con los datos recibidos.
9. El usuario ve el resultado en el navegador sin ninguna excepción visible.

Flujo de una petición con error controlado:

10. Usuario pulsa 'Pokemon inexistente (404)' en el simulador.
11. POST a Spring Boot: /api-simulator/llamar con endpoint=/api/pokemon-error.
12. ApiSimulatorController llama a http://localhost:5000/api/pokemon-error.
13. Flask llama a PokeAPI con el nombre 'pokemonquenuncaexistira99999'.
14. PokeAPI devuelve 404. Flask captura HTTPError y devuelve JSON de error con HTTP 404.
15. Spring Boot recibe el 404. RestTemplate lanza HttpClientErrorException.
16. El bloque catch(HttpClientErrorException) captura la excepción.
17. Se llama a traducirError4xx() que devuelve mensaje en español para el código 404.
18. El modelo se puebla con éxito=false, tipoExcepcion y mensajeError traducido.
19. Thymeleaf renderiza el panel rojo con el mensaje traducido, sin ningún stacktrace.

Flujo cuando Flask no está disponible:

20. Docker está parado o el contenedor sisdis_flask está detenido.
21. Usuario invoca cualquier endpoint del simulador.
22. RestTemplate intenta la conexion a localhost:5000 y esta es rechazada.
23. RestTemplate lanza ResourceAccessException (Connection refused).
24. El bloque catch(ResourceAccessException) captura la excepción.
25. Se muestra al usuario: 'No se pudo conectar con la API Python. Está Docker corriendo?'
26. La aplicación Spring Boot NO se cae. Continúa funcionando normalmente.

10. Guía de Arranque Completa

> **NOTA:** Requisitos previos: Docker Desktop instalado y ejecutándose, IntelliJ IDEA con el proyecto abierto, conexion a internet para imágenes Docker y acceso a PokeAPI.

PASO 1 - Verificar que Docker Desktop está corriendo

Abre Docker Desktop. Espera a que el icono de la barra de tareas de Windows este en verde con el texto 'Engine running'. Si no está verde, ábrelo y espera 30-60 segundos.

PASO 2 - Abrir terminal en la carpeta del proyecto

En el Explorador de Windows, navega hasta la carpeta raíz del proyecto (donde esta docker-compose.yml). Haz clic en la barra de direcciones, escribe 'cmd' y pulsa Enter.

PASO 3 - Levantar Docker (PostgreSQL + Flask)

```
docker compose up -d --build
```

El flag --build construye la imagen de Flask desde el Dockerfile. La primera vez tarda un poco descargando las imágenes. En ejecuciones posteriores con usar *docker compose up -d* funciona.

PASO 4 - Verificar contenedores activos

```
docker compose ps
```

Debes ver los dos contenedores en estado running:

```
NAME                STATUS
sisdis_postgres     running (healthy)
sisdis_flask        running
```

PASO 5 - Verificar que Flask responde

```
http://localhost:5000/health
```

Debes ver en el navegador:

```
{"servicio": "Flask API - SisDis Practica 2", "status": "ok"}
```

PASO 6 - Arrancar Spring Boot desde IntelliJ

Con Docker corriendo, ve a IntelliJ. Abre Practica2Application.java y pulsa el boton verde de Run (Shift+F10). Espera a que la consola muestre:

```
Usuarios creados: admin/admin123 | usuario/user123
Started Practica2Application in X.XXX seconds
```

PASO 7 - Verificar la aplicacion completa

URL	Resultado esperado
http://localhost:8080	Página principal con hero oscuro y tarjetas de features
http://localhost:8080/login	Formulario de login con diseño Pokédex
Login con admin/admin123	Redirige a /dashboard tras login correcto
http://localhost:8080/dashboard	Panel con 6 tarjetas de escenarios de excepción
http://localhost:8080/api-simulator	Simulador con 4 grupos de botones

Comandos Docker útiles:

```
# Ver logs de Flask en tiempo real
docker compose logs -f python-api

# Parar todo (conserva datos)
docker compose down

# Parar y eliminar datos (reset completo)
docker compose down -v

# Reconstruir Flask tras cambiar app.py
docker compose up -d --build python-api

# Reiniciar solo Flask
docker compose restart python-api
```

11. Pruebas con Postman

Postman es una herramienta grafica para probar APIs REST. Permite enviar peticiones HTTP con distintos métodos, cabeceras, cuerpos y autenticación, y visualizar la respuesta con el código de estado y el cuerpo JSON.

Se crea una colección 'Practica2 - Sistemas Distribuidos' con dos carpetas: 'Flask API Directa' para probar Flask sin pasar por Spring, y 'Spring Boot a Flask' para probar el flujo completo.

Carpeta 1: Flask API Directa (localhost:5000)

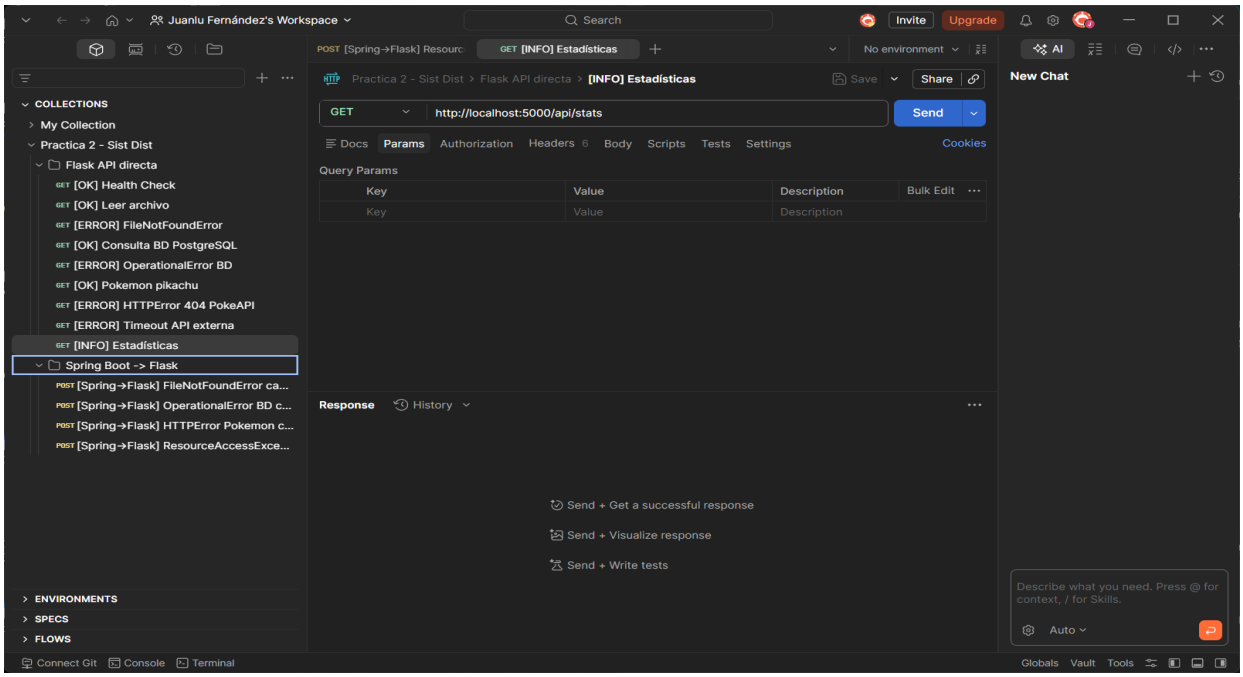
#	Nombre	Método	URL	Código esperado
1	Health Check	GET	/health	200 OK
2	Archivo OK	GET	/api/file-ok	200 OK
3	FileNotFoundError	GET	/api/file-error	404 Not Found
4	BD correcta	GET	/api/db-ok	200 OK
5	OperationalError BD	GET	/api/db-error	500 Internal Server Error
6	Pokemon OK	GET	/api/pokemon/pikachu	200 OK
7	HTTPError 404 PokeAPI	GET	/api/pokemon-error	404 Not Found
8	Timeout API externa	GET	/api/pokemon-timeout	504 Gateway Timeout
9	Estadísticas sesión	GET	/api/stats	200 OK

Carpeta 2: Spring Boot a Flask (localhost:8080)

> **NOTA:** Para estas peticiones configurar Basic Auth en Postman: pestana Auth -> tipo Basic Auth -> username: admin, password: admin123. Body: x-www-form-urlencoded con clave 'endpoint'.

#	Nombre	Método	URL	Body endpoint=	Resultado
10	FileNotFoundError capturado	POST	/api-simulator/llamar	/api/file-error	HTML panel rojo, mensaje español
11	OperationalError BD capturado	POST	/api-simulator/llamar	/api/db-error	HTML panel rojo 500

#	Nombre	Método	URL	Body endpoint=	Resultado
12	HTTPError Pokemon capturado	POST	/api-simulator/llamar	/api/pokemon-error	HTML panel rojo 404
13	ResourceAccessException (Flask caído)	POST	/api-simulator/llamar	/api/file-ok	HTML 'API no disponible'



> **NOTA:** Las capturas de todas las pruebas con Postman están en la carpeta capturas_postman en la carpeta documentación de la práctica.

12. Verificación y Comprobación del Sistema

Esta sección recoge todos los puntos de verificación que permiten confirmar que el sistema funciona correctamente en su totalidad.

12.1 Verificación de Docker

Comprobación	Comando / Acción	Resultado esperado
Contenedores activos	docker compose ps	sisdis_postgres (healthy) y sisdis_flask running
Health check Flask	GET http://localhost:5000/health	{"status": "ok"}
Logs Flask	docker compose logs python-api	Líneas INFO sin errores críticos
Conexion BD desde Flask	GET http://localhost:5000/api/db-ok	Versión de PostgreSQL 16.x

12.2 Verificación del Frontend Spring Boot

Comprobación	Acción	Resultado esperado
Página principal	Abrir http://localhost:8080	Hero oscuro, 3 feature cards, botón de acceso

Comprobación	Acción	Resultado esperado
Login correcto	Credenciales admin/admin123	Redirige a /dashboard
Login incorrecto	Credenciales erróneas	Permanece en login con mensaje de error en rojo
Logout	Pulsar 'Salir' en la navbar	Redirige a /login con mensaje de confirmación verde
Acceso sin autenticar	Ir a /dashboard sin sesion	Redirige automáticamente a /login

12.3 Verificación del Simulador de Excepciones

Botón	Excepción demostrada	Resultado visible en el navegador
Archivo OK	Ninguna (éxito)	Panel verde con contenido del archivo
FileNotFoundError	FileNotFoundError (Python)	Panel rojo: 'Recurso no encontrado (404)...'
BD OK	Ninguna (éxito)	Panel verde con versión de PostgreSQL
Error BD	psycopg2.OperationalError	Panel rojo: 'Error interno servidor Python (500)...'
Pokemon OK	Ninguna (éxito)	Panel verde con datos de pikachu (nombre, ID, peso, altura)
Pokemon ERROR	requests.HTTPError	Panel rojo: 'Recurso no encontrado (404)...'
Timeout	requests.Timeout	Panel rojo: 'Timeout al llamar a PokeAPI... (504)'
Estadísticas	Ninguna (info)	Panel verde con JSON de contadores por endpoint
Flask parado	ResourceAccessException	Panel rojo: 'No se pudo conectar con la API Python'

12.4 Verificación de Logs en IntelliJ

```
# Invocacion exitosa
INFO ApiSimulatorController : >>> Invocando API Python:
http://localhost:5000/api/db-ok
INFO ApiSimulatorController : <<< Respuesta OK: HTTP 200

# Error 4xx
WARN ApiSimulatorController : <<< Error 4xx en .../api/pokemon-error: 404

# Error 5xx
ERROR ApiSimulatorController : <<< Error 5xx en .../api/db-error: 500

# Flask no disponible
ERROR ApiSimulatorController : <<< API Python NO disponible: Connection refused
```

12.5 Verificación de Logs en Docker (Flask)

```
# Ejecutar: docker compose logs -f python-api
2025-01-15 10:23:45 [INFO] Invocado /api/file-ok
2025-01-15 10:23:46 [INFO] Invocado /api/file-error -- simulando FileNotFoundError
2025-01-15 10:23:47 [INFO] Invocado /api/db-ok
2025-01-15 10:23:49 [INFO] Invocado /api/pokemon/pikachu
127.0.0.1 - [GET /api/file-ok] 200 -
127.0.0.1 - [GET /api/file-error] 404 -
127.0.0.1 - [GET /api/db-error] 500 -
```

13. Conclusiones

La Practica 2 de Sistemas Distribuidos ha permitido implementar un sistema distribuido real y funcional que demuestra de forma práctica los conceptos fundamentales de la asignatura: comunicación entre servicios, gestión de excepciones en cascada y orquestación de entornos con Docker.

Logros técnicos alcanzados:

- Arquitectura de tres capas con Spring Boot, Flask y PostgreSQL, con comunicación HTTP entre ellas y persistencia de datos real.
- Los tres tipos de excepciones del enunciado: errores de archivos (FileNotFoundError, PermissionError), errores de base de datos (psycopg2.OperationalError, DatabaseError) y errores de APIs externas (requests.HTTPError, Timeout, ConnectionError).
- Ninguna excepción llega sin tratar al usuario. Todas se capturan, clasifican y traducen al español antes de mostrarse.
- Sistema resiliente: si la API Flask no está disponible, Spring Boot captura la ResourceAccessException y continúa funcionando.
- Autenticación completa con Spring Security, logging en ambas capas y estadísticas de uso de la API Flask.
- Entorno completamente reproducible con Docker Compose: cualquier maquina con Docker puede levantar el sistema con un solo comando.

Aprendizajes de la practica:

- La importancia de estandarizar los errores en las APIs REST: devolver siempre JSON con campos tipo y detalle permite al consumidor procesar el error de forma programática.
- El patrón ControllerAdvice de Spring es fundamental para el manejo centralizado de excepciones, evitando código repetido en cada controlador.
- Docker Compose facilita la reproducibilidad del entorno, eliminando el problema de 'en mi maquina funciona'.
- La diferencia entre excepciones críticas y no criticas es clave: un timeout en una API de terceros no debe tumbar toda la aplicación.
- El health check de Docker es esencial cuando hay dependencias entre servicios para evitar errores de conexion en el arranque.